

Transformers

Sequence-to-Sequence Models, Attention, and the Transformer



Timing: 2 minutes

Good evening everyone. Today we are moving to one of the most important topics in modern NLP: Transformers. The full path for today is sequence-to-sequence models, attention, and finally the Transformer architecture.

You have already seen simpler NLP ideas like TF-IDF, embeddings, and language models. Today we connect those ideas to the architecture behind many modern systems, including BERT-style models and large language models.

Do not worry if some diagrams look complex at first. I will build the story slowly: first, why RNN-based sequence-to-sequence models were needed; second, why they had a bottleneck; third, how attention solved part of the problem; and fourth, how the Transformer removed recurrence and made training much more scalable.

Overview

- The technology evolution
 - What is wrong with RNN?
 - Translation framework (seq-2-seq, encoder-decoder)
 - Information bottleneck
 - “Jointly Learning To Align And Translate”
 - Attention mechanism
- Attention is all you need
- Transformer
 - encoder
 - position embedding
 - decoder
- Applications

Timing: 3 minutes

Let me first give you the roadmap. We begin with the technology evolution. The first question is: what is wrong with using a normal RNN for translation?

Then we introduce the encoder-decoder framework, also called sequence-to-sequence. This is useful whenever the input and output are both sequences, but they may have different lengths. Translation is the easiest example.

After that we discuss the information bottleneck. The encoder tries to squeeze the whole source sentence into one vector, and that becomes a problem for long sentences.

Then we introduce the paper “Neural Machine Translation by Jointly Learning to Align and Translate.” This is where attention becomes important.

Finally, we move to “Attention Is All You Need” and the Transformer: encoder, positional encoding, decoder, and applications. By the end, the key idea should be clear: attention lets tokens look at other tokens directly instead of depending only on a compressed hidden state.

Learning Objectives

- Explain why variable-length output tasks (translation, summarisation, dialogue) need an encoder-decoder architecture
- Describe the information bottleneck problem and why it limits performance on long sequences
- Explain how Bahdanau attention resolves the bottleneck by letting the decoder access all encoder states
- Interpret an attention weight matrix as a soft alignment between source and target tokens
- Explain self-attention via the Query-Key-Value metaphor and compute scaled dot-product attention
- Describe multi-head attention and positional encoding
- Trace one Transformer encoder block: self-attention -> add & norm -> feedforward -> add & norm
- Articulate why the Transformer's parallelism makes it more scalable than recurrent architectures

Timing: 3 minutes

These are the learning objectives for today. By the end of this lecture, I want you to be able to explain why tasks like translation, summarisation, and dialogue need an encoder-decoder style architecture.

Second, you should be able to explain the information bottleneck: why forcing all source information into one final hidden state limits long-sequence performance.

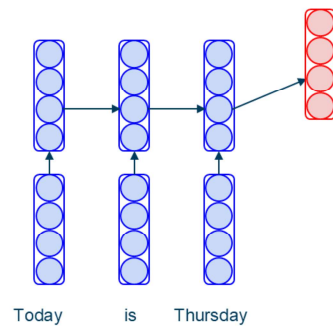
Third, you should understand the main idea of Bahdanau attention: at each decoding step, the decoder can look back at all encoder states, not only the last one.

Fourth, when you see an attention matrix, you should be able to say what the rows and columns mean, and how bright cells represent soft alignment.

Then we move to self-attention using the Query-Key-Value idea. We will cover scaled dot-product attention, multi-head attention, positional encoding, and finally one Transformer encoder block. The big takeaway is why Transformers are more scalable than recurrent models.

Using RNN to translate

- Input (source sentence) = "Today is Thursday"
- Output (target sentence) = "aujourd'hui, c'est jeudi"
- In a consecutive translation setting



Timing: 4 minutes

Let us start with a very simple translation example. The input sentence is "Today is Thursday." The target output is the French sentence "aujourd'hui, c'est jeudi."

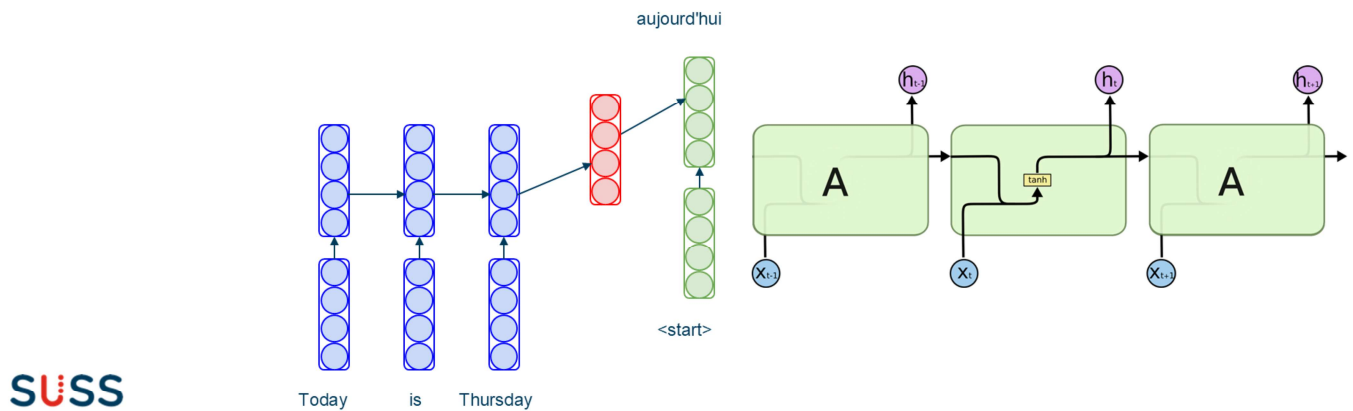
If we use an RNN to read the English sentence, it reads one word at a time: Today, then is, then Thursday. At every step, the hidden state is updated. You can think of the hidden state as the model's memory so far.

After reading the final word, the model has a final hidden state. This final hidden state is supposed to represent the meaning of the full English sentence.

At this point, the model has not generated the French sentence yet. It has only encoded the source sentence. The question is: how do we now generate a target sentence, possibly with a different length and different word order? That is why we need a decoder.

Using RNN to translate

- Input (source sentence) = "Today is Thursday"
- Output (target sentence) = "aujourd'hui, c'est jeudi"
- In a consecutive translation setting



5

Timing: 4 minutes

Now we add the decoder side. The decoder begins with a special start token. You can think of this as telling the model: now start producing the target sentence.

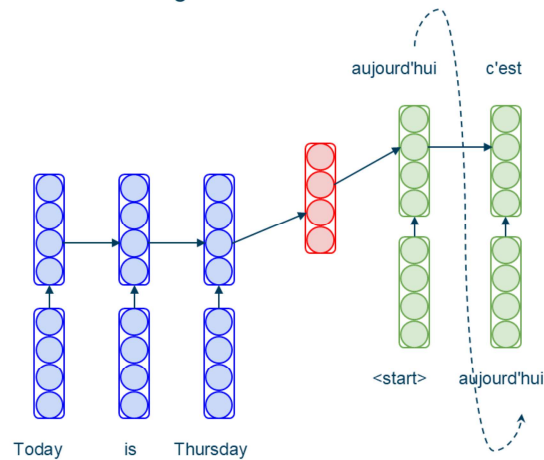
The decoder receives the context from the encoder and the start token. Based on those, it predicts the first French word: "aujourd'hui."

This is similar to a language model, but with one extra condition. A normal language model predicts the next word based only on previous target words. Here, the decoder predicts the next target word based on previous target words plus the source sentence representation.

So the first decoding step is: source sentence plus start token goes in, first output word comes out. This pattern will repeat for the next word, and then the next word, until the whole sentence is generated.

Using RNN to translate

- Input (source sentence) = "Today is Thursday"
- Output (target sentence) = "aujourd'hui, c'est jeudi"
- In a consecutive translation setting



Timing: 4 minutes

Now the decoder has already produced the first word, "aujourd'hui." To produce the next word, we feed the previous word back into the decoder.

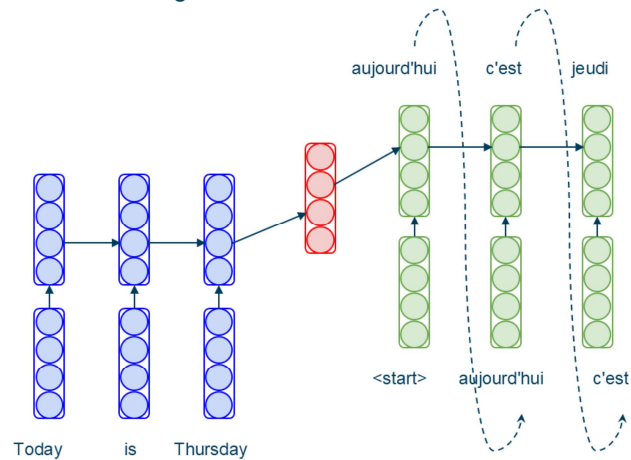
So the decoder input is now the start token plus "aujourd'hui," together with the encoded source context. The decoder predicts the next word: "c'est."

This is important because the target sentence is generated one step at a time. The model does not produce the whole French sentence in one shot. It predicts one token, then uses that token to help predict the next token.

During training, we often use the true previous word rather than the model's predicted previous word. This is called teacher forcing. We will return to this idea later, but for now remember: decoding is step-by-step generation.

Using RNN to translate

- Input (source sentence) = "Today is Thursday"
- Output (target sentence) = "aujourd'hui, c'est jeudi"
- In a consecutive translation setting



Timing: 4 minutes

The same process continues for the third word. The decoder has the start token, then "aujourd'hui," then "c'est." Based on this history and the encoded English sentence, it predicts "jeudi."

At this point, the model has produced the target sentence: "aujourd'hui, c'est jeudi." In a full system, there is also an end-of-sentence token so the model knows when to stop.

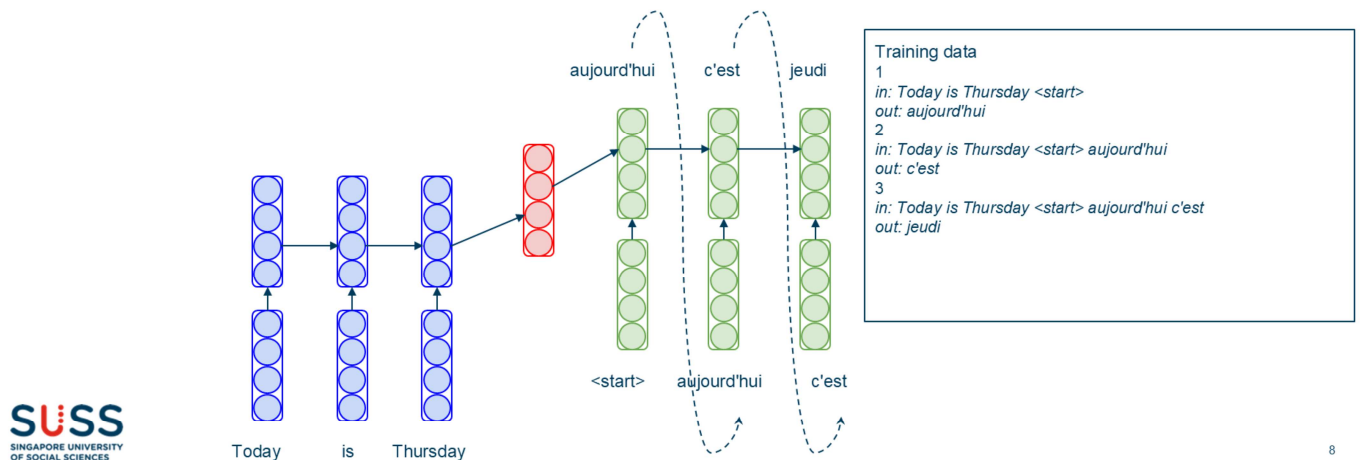
This slide is showing the key seq2seq idea: the encoder reads the source sequence, and the decoder generates the target sequence. The source and target can have different lengths. The order can also change between languages.

So the encoder-decoder model is more flexible than a simple one-input, one-output classifier. It is designed for sequence input to sequence output tasks.

Using RNN to translate

- Input (source sentence) = "Today is Thursday"
- Output (target sentence) = "aujourd'hui, c'est jeudi"
- In a consecutive translation setting

- <start> aujourd'hui, c'est
- aujourd'hui, c'est jeudi



Timing: 5 minutes

This slide connects the diagram to the training data. During training, we convert the target sentence into multiple next-word prediction examples.

Example one: input is the source sentence plus the start token. The desired output is "aujourd'hui."

Example two: input is the source sentence plus start token plus "aujourd'hui." The desired output is "c'est."

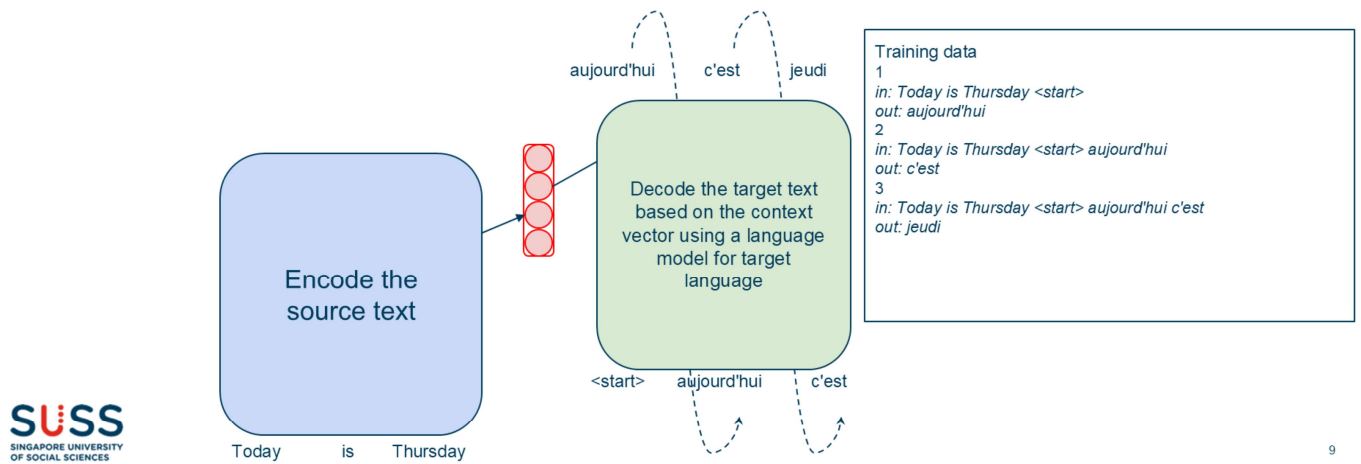
Example three: input is the source sentence plus start token plus "aujourd'hui c'est." The desired output is "jeudi."

This is how the decoder learns to generate a full sequence. It learns many small next-token prediction tasks.

A useful way to think about this is: translation is broken into repeated next-word prediction, but every prediction is conditioned on the source sentence. That is the difference from a standalone language model.

Using RNN to translate

- Input (source sentence) = "Today is Thursday"
- Output (target sentence) = "aujourd'hui, c'est jeudi"
- In a consecutive translation setting



Timing: 4 minutes

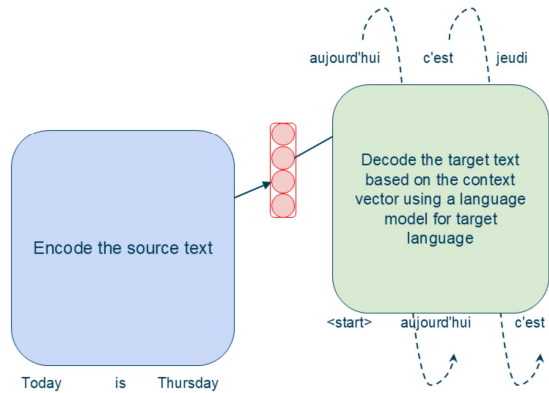
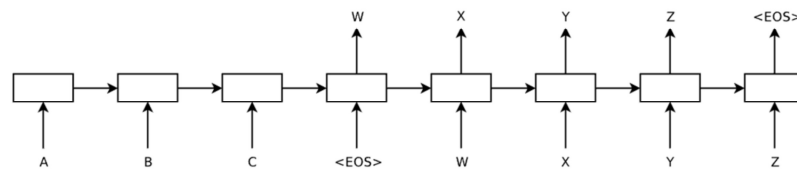
Now we abstract the picture. On the left, the encoder reads the source text. It compresses the source information into a context vector. On the right, the decoder uses that context vector and behaves like a target-language model.

For our example, the source text is "Today is Thursday." The target-language model generates the French words one by one.

The important word here is "context vector." In the original encoder-decoder design, the decoder mainly depends on this fixed-size vector from the encoder.

This design is elegant and useful, but it creates a serious problem. If the sentence is short, one vector may be enough. But if the source sentence is long, detailed information from early words may be lost. This is the bottleneck that motivates attention.

Sequence to Sequence task



1. text to speech
2. speech to text
3. machine translation
4. summarization
5. dialogue system / chatbot
6. image generation
7. video generation

Timing: 5 minutes

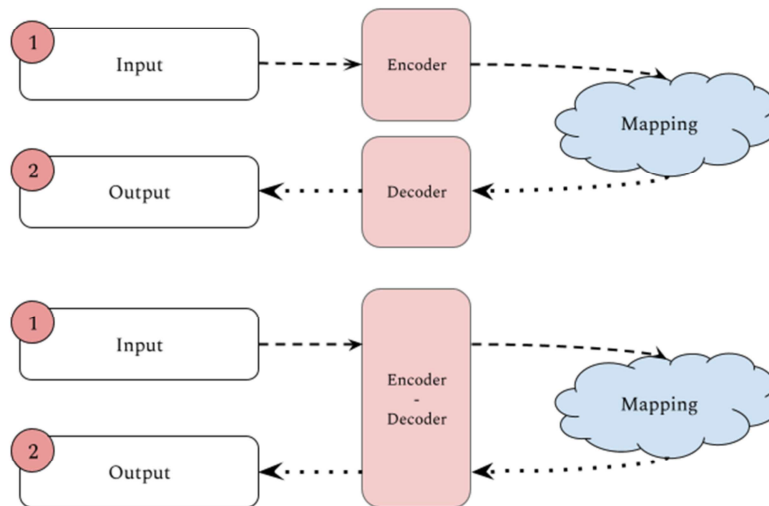
This slide generalises the idea of sequence-to-sequence tasks. Machine translation is the classic example, but the same pattern appears in many areas.

In text-to-speech, the input is text and the output is audio. In speech-to-text, the input is audio and the output is text. In summarisation, the input may be a long article and the output is a shorter summary.

A dialogue system or chatbot also works in this broad setting: the input is the conversation history and the output is the next response. Image generation and video generation can also be framed as sequence generation, depending on how the data is represented.

The key point is that the output length is not fixed. We are not just predicting one class label. We are producing a structured sequence. That is why the encoder-decoder idea became so important.

Encoder-decoder framework



Timing: 3 minutes

This slide shows the encoder-decoder framework at a higher level. The encoder maps the input into an internal representation. The decoder maps that representation into the output.

In some tasks, the encoder and decoder may be separate modules. In other tasks, they may be parts of a larger model. But the basic idea is the same: one part understands or represents the input, and another part produces the output.

For translation, the encoder understands the source language sentence, and the decoder generates the target language sentence.

This framework is not only for NLP. The same thinking appears in autoencoders, image captioning, speech systems, and many multimodal models. Today we focus on NLP because it gives us a clear path to attention and Transformers.

Encoder-Decoder, Formally

- Encoder: a bidirectional RNN reads $x_1, x_2, \dots, x_n$ and produces hidden states $h_1, h_2, \dots, h_n$
- The encoder's final hidden state h_n is passed to the decoder as its initial state
- Decoder: a unidirectional RNN generates $y_1, y_2, \dots, y_m$ one step at a time, conditioning on its current state and the previous output token
- Teacher forcing during training: feed the true target token at each decoder step instead of the model's own previous output — speeds convergence, but can cause exposure bias at inference time
- Anchor moment: draw this with a 4-word source and a 3-word target. Where is the bottleneck? The single vector h_n that must carry all source information into the decoder.

Timing: 5 minutes

Let us describe the encoder-decoder model more formally. The encoder is usually a bidirectional RNN. It reads tokens x_1 to x_n and produces hidden states h_1 to h_n .

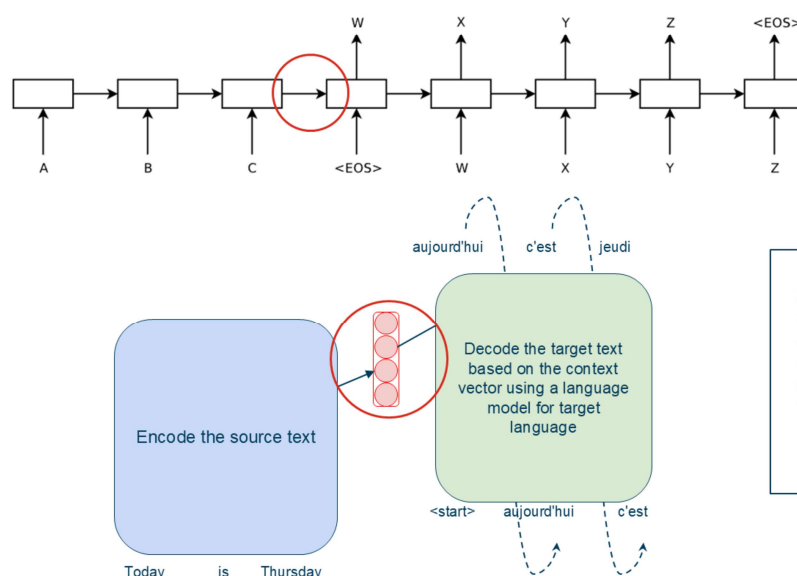
A bidirectional encoder can read information from both directions, so each hidden state can contain left and right context.

In the basic model, the final hidden state h_n is passed to the decoder as the initial state. The decoder is usually unidirectional because it generates y_1 to y_m from left to right. At each step it conditions on its current state and the previous output token.

During training, teacher forcing means we feed the true previous target token, not the model's previous prediction. This makes training easier, but during inference the model must use its own generated tokens. That difference can cause exposure bias.

The anchor question is: where is the bottleneck? It is the single vector h_n . That one vector must carry all source information into the decoder.

The information bottleneck



Timing: 4 minutes

This slide visualises the information bottleneck. The red circled point is the narrow passage between the encoder and decoder.

The encoder reads the full source sentence, but the decoder receives only a fixed-size context vector. So no matter whether the source sentence has 5 words or 50 words, the decoder is asked to rely on the same limited channel.

For short sentences like "Today is Thursday," this may work reasonably well. But for longer sentences, the final hidden state may forget or weaken information from earlier words.

Think of it like asking someone to read a long paragraph and then summarise every detail using only one small note card. Some important details will be lost. Attention was introduced to avoid forcing all information through this single note card.

The Bottleneck Problem

- The encoder's final hidden state h_n must represent the entire source sentence
 - Feasible for short sentences (5–10 words)
 - For long sentences (50+ words), early words' information is largely overwritten by the time the encoder reaches the last word
- A single fixed-size vector cannot compress arbitrarily long sequences without loss
- Empirically: vanilla encoder-decoder BLEU score drops significantly for sentences longer than ~20 words, before attention is added

Timing: 5 minutes

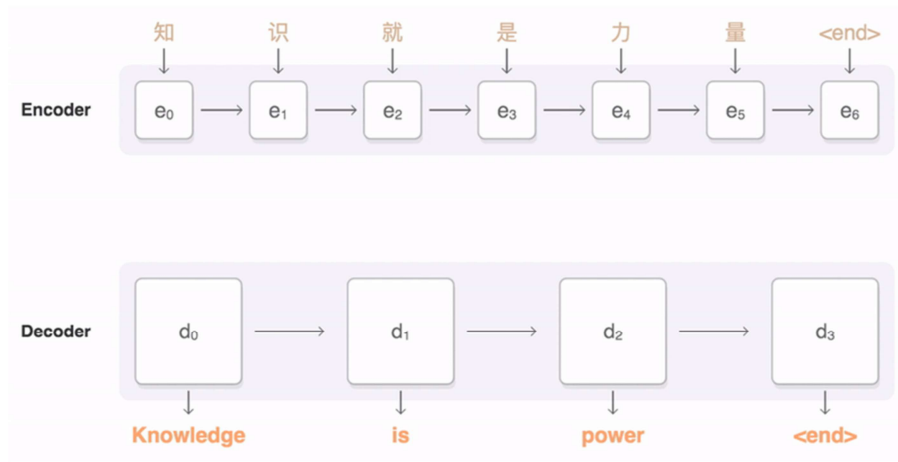
Here we state the bottleneck problem directly. The encoder's final hidden state must represent the entire source sentence. This is feasible for short sentences, maybe five to ten words.

But for long sentences, especially fifty words or more, early information can be overwritten as the RNN processes later words. The hidden state is updated repeatedly, so it has to decide what to keep and what to forget.

A single fixed-size vector cannot compress arbitrarily long sequences without loss. This is not only a theory problem; empirically, vanilla encoder-decoder models perform worse on longer sentences before attention is added.

So the problem is not that RNNs are useless. The problem is that the communication channel between encoder and decoder is too narrow. We need the decoder to access more than just the final state.

Attentive seq2seq



https://upload.wikimedia.org/wikipedia/commons/3/37/Seq2seq_with_RNN_and_attention_mechanism.gif

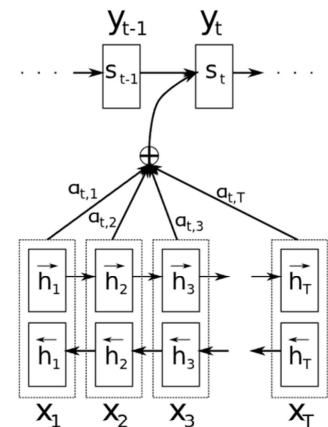


Figure 1: The graphical illustration of the proposed model trying to generate the t -th target word y_t given a source sentence $(x_1, x_2, \dots, x_T)$.

Timing: 4 minutes

This brings us to attentive sequence-to-sequence models. The important paper here is “Neural Machine Translation by Jointly Learning to Align and Translate.”

The word “align” is important. In translation, some source words correspond to particular target words, but the correspondence may not be one-to-one and the order may change.

The attention mechanism allows the decoder, at each output step, to decide which source positions are most relevant. Instead of relying only on one final context vector, the decoder can look back at all encoder hidden states.

The picture on the right shows the decoder state connected to multiple encoder hidden states with attention weights. These weights are learned by the model. The model learns where to look while generating each target word.

Bahdanau Attention (2015) — The Mechanism

- Insight: instead of compressing all source info into one vector, let the decoder look back at all encoder hidden states at each decoding step
- At decoder step t :
 - 1. Alignment scores: $e_{ti} = \text{score}(s_{(t-1)}, h_i)$ — how relevant is encoder state h_i to decoder state $s_{(t-1)}$?
 - 2. Attention weights: $\alpha_{ti} = \text{softmax}(e_t)$ — normalise scores to sum to 1
 - 3. Context vector: $c_t = \sum_i \alpha_{ti} h_i$ — weighted sum of all encoder hidden states
 - 4. Decoder state update: $s_t = f(s_{(t-1)}, y_{(t-1)}, c_t)$

Timing: 6 minutes

Now let us go through Bahdanau attention step by step. The main insight is simple: do not compress all source information into one vector. Let the decoder look back at all encoder hidden states at each decoding step.

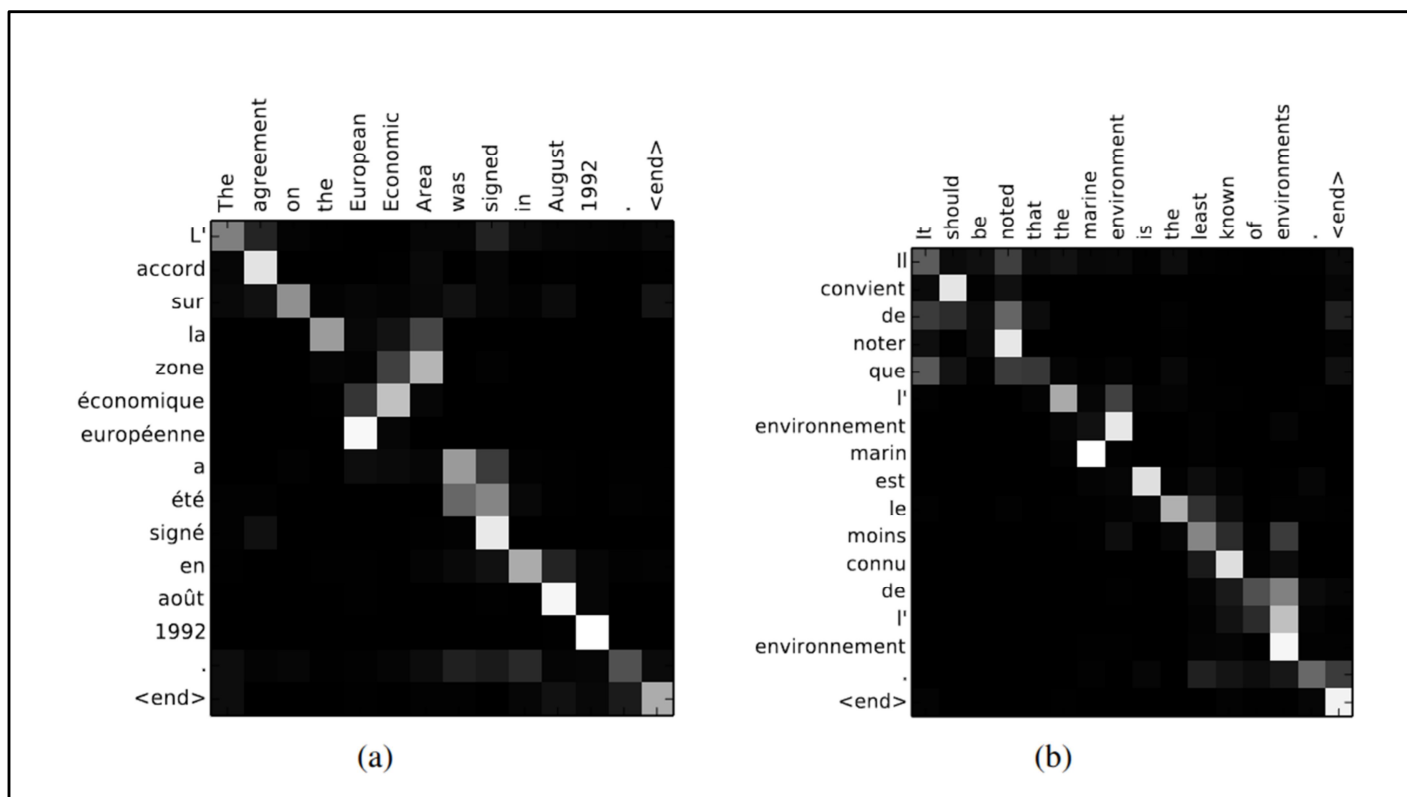
At decoder step t , the model first computes alignment scores. The score e_{ti} asks: how relevant is encoder state h_i to the previous decoder state $s_{(t-1)}$?

Then we apply softmax to these scores. This gives attention weights α_{ti} . The weights sum to 1, so they behave like a distribution over source positions.

Next, the context vector c_t is computed as a weighted sum of all encoder hidden states. If a source word is important at this decoding step, its hidden state receives a higher weight.

Finally, the decoder updates its state using the previous state, the previous output token, and this context vector.

So the context vector is no longer fixed for the whole sentence. It changes at every target word.



Timing: 5 minutes

This slide shows attention matrices from translation examples. Do not worry about every word. Focus on the pattern.

Each row corresponds to a target word being generated. Each column corresponds to a source word. A brighter cell means the decoder is paying more attention to that source word at that generation step.

In many translation examples, we see a diagonal pattern because the word order is often similar between source and target. But not always. Sometimes the bright cells move away from the diagonal because the translation requires reordering.

This is one reason attention became popular. It improved performance, and it also gave us a way to inspect what the model was using. It is not a perfect explanation, but it gives useful intuition about alignment.

Reading the Attention Matrix

- Rows = target (French) words being generated; columns = source (English) words being attended to
- Brighter cell = higher attention weight
- Roughly diagonal pattern = word order is mostly preserved between the two languages
- Off-diagonal brightness = reordering: "European Economic Area" (English order) vs "zone économique européenne" (French order) — the model attends out of order to produce the correct French phrase
- This is the kind of analysis students do in the lab

Timing: 4 minutes

Here is how to read an attention matrix carefully. Rows are target words, meaning the words being generated by the decoder. Columns are source words, meaning the words being attended to by the decoder.

A brighter cell means a higher attention weight. If the bright cells roughly follow a diagonal line, the word order is mostly preserved between the two languages.

If we see brightness away from the diagonal, that suggests reordering. For example, "European Economic Area" in English becomes "zone économique européenne" in French. The model may need to attend to source words out of the original order to produce the correct phrase.

This is exactly the type of reasoning students may do in the lab. The goal is not just to run code, but to interpret the attention pattern.

What Attention Achieves — and Its Remaining Limitation

- Removes the bottleneck — the decoder can directly access any encoder hidden state
- Provides soft alignment — learns which source words correspond to which target words
- Improves performance on long sequences significantly
- Offers some interpretability — the attention matrix can be visualised
- Remaining limitation: seq2seq with attention is still sequential — the encoder reads one token at a time
 - Training is slow, GPU utilisation is low — this motivates the Transformer
- Forward-looking question: what if every encoder token also attended to every other encoder token, not just the decoder attending to the encoder? → self-attention

Timing: 4 minutes

Let us summarise what attention achieves. First, it removes the strict bottleneck. The decoder can directly access any encoder hidden state instead of depending only on the final state.

Second, it provides soft alignment. The model learns which source words are useful for generating each target word.

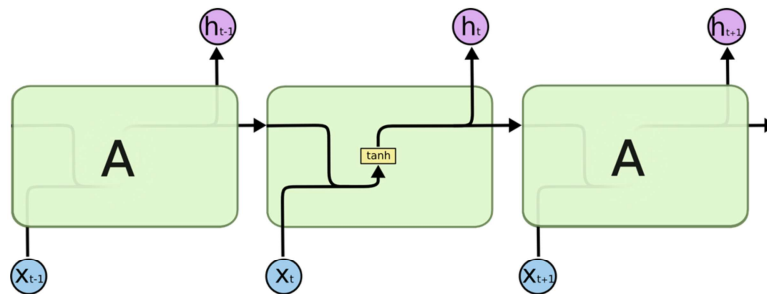
Third, it improves performance on long sequences because early source information remains accessible. Fourth, the attention matrix gives some interpretability.

However, there is still one major limitation. This model is still based on RNNs. The encoder reads the source sentence one token at a time, and the decoder generates one token at a time.

So attention improves memory access, but it does not fully solve the speed problem. This motivates the Transformer.

RNN is slow...

- RNN is slow because of recurrence mechanism ($\text{speed} \propto \text{seq_len}$)
- Attention Is All You Need
 - dispensing with recurrence and convolutions entirely
 - requiring significantly less time to train



Timing: 4 minutes

The next problem is speed. RNNs are slow because of recurrence. Token 2 depends on token 1, token 3 depends on token 2, and so on.

This means we cannot process all positions fully in parallel. For long sequences, the computation path grows with sequence length. GPUs are very good at parallel matrix operations, but RNNs force a sequential dependency.

The paper “Attention Is All You Need” made a bold proposal: remove recurrence and convolution entirely, and rely on attention mechanisms.

This is a major reason Transformers became dominant. They process all positions in a sequence simultaneously during training. That makes much better use of GPUs and allows models to scale to much larger data and parameter sizes.

From Attention to Self-Attention

- Recap: the decoder attended to encoder hidden states — "which source positions are relevant to what I'm generating now?"
- New question: apply the same idea within a single sequence — every token attends to every other token in the same sequence
- Why this is powerful: "The animal didn't cross the street because it was too tired." — self-attention lets "it" compute attention weights over "animal" and "street" to resolve the reference
- The Query-Key-Value abstraction (library catalogue metaphor):
 - Query (Q): what you are looking for
 - Key (K): the index entries in the catalogue — what each item offers
 - Value (V): the actual content retrieved when an item is selected

Timing: 6 minutes

Now we move from attention to self-attention. In the previous model, the decoder attended to encoder hidden states. The question was: which source positions are relevant to what I am generating now?

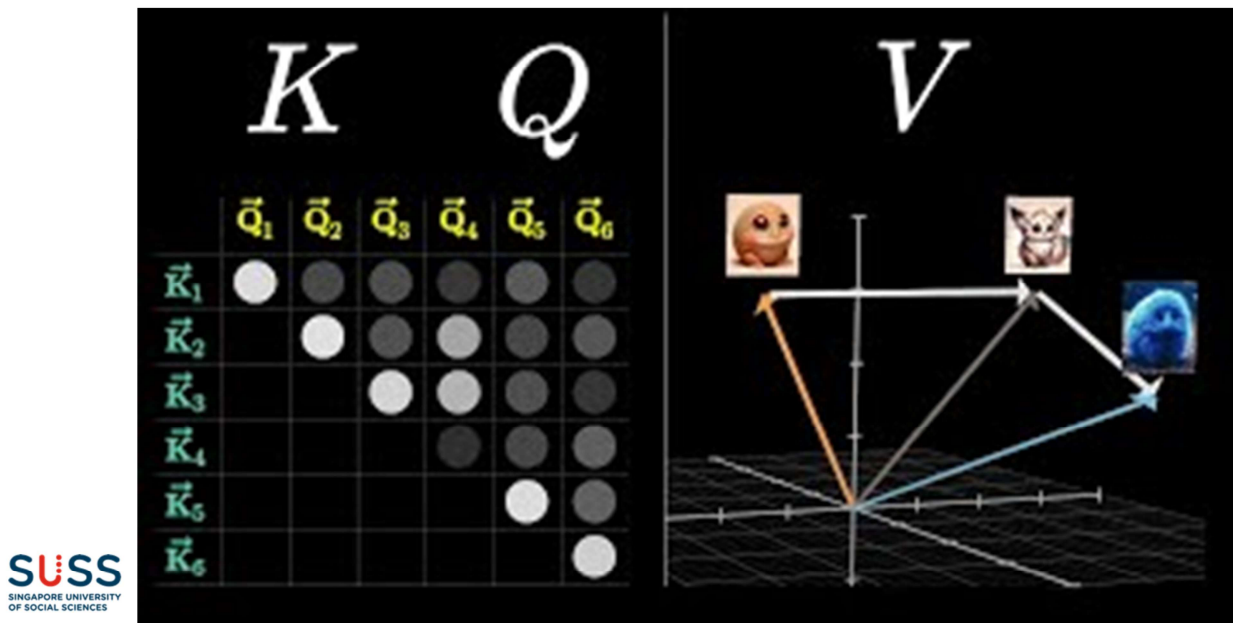
Self-attention applies a similar idea within a single sequence. Every token can attend to every other token in the same sequence.

Consider this sentence: "The animal didn't cross the street because it was too tired." The word "it" probably refers to "animal," not "street." Self-attention allows the representation of "it" to look at other words and assign higher relevance to the right one.

The Query-Key-Value metaphor helps. Query means what I am looking for. Key means what each item offers, like an index entry in a catalogue. Value is the actual content retrieved.

So each token asks a query, compares it with keys from all tokens, and combines the values according to relevance.

Transformer



Timing: 4 minutes

This Transformer slide shows the Query, Key, and Value idea visually. The model computes attention by comparing queries and keys, and then using the resulting weights to combine values.

In simple terms, a token asks: what information do I need? That is the query. Other tokens advertise what they contain. Those are the keys. Once the model decides which tokens are relevant, it pulls useful information from the values.

The important shift from RNNs is that this happens across the whole sequence in parallel. A token does not need to wait for information to travel step by step through many recurrent states.

This is why self-attention is powerful for long-range dependencies. If two words are far apart but semantically connected, self-attention can connect them directly.

Self-Attention, Formally

- For each position i : $Q_i = x_i W^Q$, $K_j = x_j W^K$, $V_j = x_j W^V$
- 1. Raw alignment score: $e_{ij} = Q_i \cdot K_j$ (dot product — higher = more relevant)
- 2. Scale: $e_{ij} / \sqrt{d_k}$ — prevents softmax saturation at large values
- 3. Normalise: $\alpha_{ij} = \text{softmax}_j(e_{ij} / \sqrt{d_k})$
- 4. Aggregate: $z_i = \sum_j \alpha_{ij} V_j$ — weighted sum of all values
- Why divide by $\sqrt{d_k}$? The dot product grows with d_k (sum of d_k terms) — large values push softmax into near-zero-gradient regions
- Anchor moment: work through a toy 2D example on the board — 3 tokens, compute attention weights and the output for one of them

Timing: 6 minutes

Let us write self-attention more formally. For each token position, we create three vectors: Q , K , and V . These are produced by multiplying the token representation by learned weight matrices W^Q , W^K , and W^V .

First, we compute a raw alignment score by taking the dot product between a query Q_i and a key K_j . A higher dot product means higher relevance.

Second, we divide by square root of d_k . This scaling prevents the dot products from becoming too large when the vector dimension is large. If values are too large, softmax becomes extremely sharp and gradients become very small.

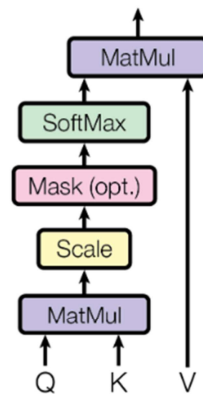
Third, we apply softmax across positions to get attention weights.

Fourth, we compute z_i as the weighted sum of value vectors. This z_i is the new representation for token i after it has collected information from the sequence.

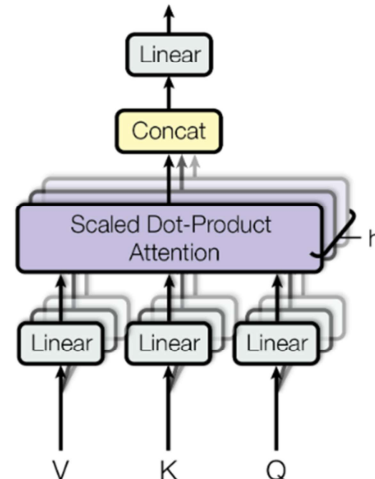
The core formula is: compare with keys, normalise, then combine values.

Attention head

Scaled Dot-Product Attention



Multi-Head Attention



Timing: 5 minutes

This slide shows the attention head from the Transformer paper. On the left is scaled dot-product attention. Q and K are multiplied to produce scores. The scores are scaled, optionally masked, passed through softmax, and then multiplied with V to produce the output.

The optional mask is important in decoders. During text generation, the model should not look at future tokens. So masking prevents a position from attending to tokens that come after it.

On the right is multi-head attention. Instead of doing attention once, the model does it several times in parallel, using different learned projections.

Each head can learn a different relationship. After the heads finish, their outputs are concatenated and passed through a linear layer. This gives the model richer ways to represent context.

Multi-Head Attention

- One attention head learns one way to compute relevance — but a word relates to context in multiple ways at once:
 - "bank" near "loan" — financial relationship
 - "bank" as subject of "flooded" — physical/geographic relationship
 - "bank" preceding "on Tuesday" — temporal relationship
- Multi-head attention runs h parallel heads, each with its own W^Q , W^K , W^V
 - Outputs concatenated and projected: $\text{concat}(\text{head}_1, \dots, \text{head}_h) \times W^O$
- Typical: $h = 8$ or 12 heads, each of dimension $d_k = d_{\text{model}} / h$

Timing: 4 minutes

Why do we need multiple heads? One attention head can learn one way of measuring relevance. But language has many relationships at the same time.

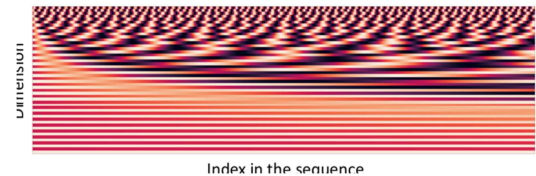
Take the word “bank.” Near “loan,” it may have a financial meaning. As the subject of “flooded,” it may refer to a river bank. Near a time phrase like “on Tuesday,” a head might capture temporal information.

Multi-head attention lets the model represent several such relationships in parallel. Each head has its own WQ , WK , and WV matrices.

The outputs of all heads are concatenated and projected. Typically, models use heads such as 8 or 12, and each head has a smaller dimension than the full model dimension.

The simple takeaway: different heads can look for different types of relationships in the same sequence.

- Self-attention as defined is permutation-invariant: "cat sat mat" and "mat sat cat" give the same computation — scores depend only on $Q \cdot K$, not position
- **Positional Encoding**
We need to inject position information
- Sinusoidal positional encodings (Vaswani et al. 2017) are added to word embeddings before the first Transformer block
- Anchor moment: if we removed positional encoding, "The dog bit the man" and "The man bit the dog" would produce identical output — completely wrong



Timing: 5 minutes

Now we need to solve another issue. Self-attention by itself does not know word order. The computation depends on similarities between Q and K vectors, not directly on position.

If we only use token embeddings, the model may not distinguish "cat sat mat" from "mat sat cat" properly, because the set of words is the same. But order clearly matters in language.

So Transformers add positional encoding to word embeddings before the first Transformer block. In the original Transformer, sinusoidal positional encodings were used.

A simple example: "The dog bit the man" and "The man bit the dog" contain the same words, but the meaning is completely different. Without position information, the model cannot reliably represent this difference.

So token meaning plus token position gives the model enough information to reason about sequence order.

The encoder decoder

<https://bbycroft.net/llm> for visualization

<https://github.com/hyunwoongko/transformer> for vanilla transformer implementation

https://huggingface.co/docs/transformers/main/en/model_doc/bert#transformers.BertModel for actual implementation

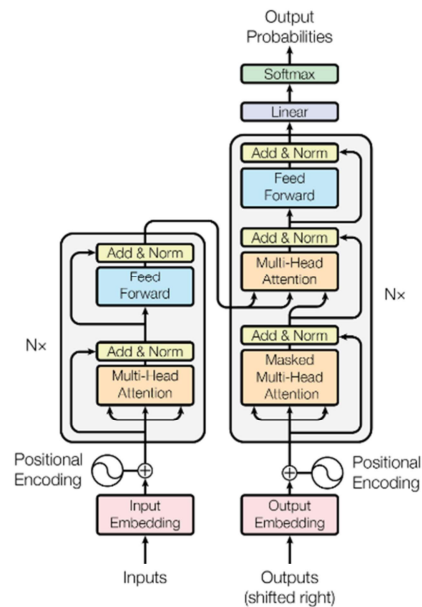


Figure 1: The Transformer - model architecture.

27

Timing: 3 minutes

This slide points to useful resources. The first link is a visualisation that helps you explore how Transformer components work. Visual tools are very helpful here because the architecture can feel abstract from equations alone.

The second link is a vanilla Transformer implementation. This is useful if you want to see how the architecture is written from scratch, with clear building blocks.

The third link points to the Hugging Face BERT model documentation. This is closer to what you will use in real projects: a production-grade library where the architecture is already implemented and we fine-tune it for downstream tasks.

The diagram on the right is the full Transformer architecture. Today we are mainly building the intuition for the encoder side and the attention mechanism.

The Transformer Encoder Block

- Each block has two sub-layers:
 - 1. Multi-head self-attention + residual connection + layer normalisation
 - 2. Position-wise feedforward network + residual connection + layer normalisation
- $X' = \text{LayerNorm}(X + \text{SelfAttn}(X))$; $X'' = \text{LayerNorm}(X' + \text{FFN}(X'))$
- Residual connections: let gradients flow directly back without passing through the full sub-layer computation — critical for training deep stacks
- Layer normalisation: normalises each token's representation to mean 0, variance 1 across dimensions — makes training more stable

Timing: 5 minutes

Let us focus on one Transformer encoder block. Each encoder block has two main sub-layers.

The first sub-layer is multi-head self-attention. This lets each token gather information from other tokens. After that, we add a residual connection and apply layer normalisation.

The second sub-layer is a position-wise feedforward network. This is applied to each token representation independently. Again, we add a residual connection and apply layer normalisation.

The equations show this compactly. First: X' equals $\text{LayerNorm}(X + \text{SelfAttention}(X))$. Second: X'' equals $\text{LayerNorm}(X' + \text{FFN}(X'))$.

Residual connections help gradients flow through deep networks. Layer normalisation stabilises training. Without these, training deep Transformer stacks would be much harder.

Why the Transformer Dominates

- 1. Parallelism: all positions processed simultaneously — unlike RNNs, GPUs are fully utilised
- 2. Long-range dependencies: every token attends to every other token in $O(1)$ operations, vs. $O(n)$ recurrent steps for RNNs
- 3. Scalability: adding parameters consistently improves performance; RNNs have diminishing returns from the hidden-state bottleneck
- 4. Pre-training efficiency: MLM/next-token objectives use every position as a supervision signal simultaneously

Timing: 4 minutes

Why did the Transformer dominate? The first reason is parallelism. All positions can be processed simultaneously during training, unlike RNNs where positions depend on previous positions. This uses GPUs much more efficiently.

Second, self-attention handles long-range dependencies well. Any token can attend to any other token directly. In an RNN, information may need to pass through many recurrent steps.

Third, Transformers scale well. When we increase data, parameters, and compute, performance tends to improve strongly. RNNs struggled more with hidden-state bottlenecks and sequential computation.

Fourth, pre-training is efficient. Objectives like masked language modelling or next-token prediction provide supervision across many positions.

So the Transformer is not just a small architectural change. It changed how large-scale NLP systems are trained.

TMA Q5 and the Course Arc

- TMA Q5: identify the shared limitation of TF-IDF, GloVe avg, bigram LM, and GRU, then explain what architectural feature addresses it
- All four have a fixed or bounded context — no mechanism lets any token directly access any other token at arbitrary distance without a bottleneck
- Attention: computes a relevance-weighted sum over all other token representations without that bottleneck
- The arc: n-gram LM (counts, limited window) → neural LM (learned representations, sequential) → seq2seq + attention (bottleneck removed, still sequential) → Transformer (recurrence removed, full parallelism)
- ECA B2/B3: fine-tuning a transformer and comparing it to your baseline uses exactly this understanding

Timing: 4 minutes

Now connect this lecture to the course assessment. TMA Q5 asks you to identify the shared limitation of TF-IDF, GloVe average, bigram language models, and GRUs, then explain what architectural feature addresses it.

The shared issue is context limitation. TF-IDF and GloVe averaging lose word order and detailed relationships. Bigram language models have a very small fixed window. GRUs process sequentially and still compress information through hidden states.

Attention addresses this by computing relevance-weighted sums over token representations. It gives tokens a way to access other tokens directly, rather than relying only on a fixed or narrow context.

The course arc is: n-gram language models, then neural language models, then seq2seq with attention, then Transformer. The Transformer removes recurrence and gives full parallelism.

Connections to Assessment

- Quiz 4 (this session) — sample questions:
 - Why divide dot products by $\sqrt{d_k}$? To prevent large values from pushing softmax into near-zero-gradient regions
 - Correct Transformer encoder block order? Self-attention + residual + norm, then feedforward + residual + norm
 - Why does a Transformer need positional encoding? Self-attention is permutation-invariant and the embedding layer adds no position info
- TMA due at the end of this fortnight — Q5 should now be completable
- ECA (released after this fortnight): ECA B2 requires fine-tuning distilbert-base-uncased — understanding this lecture's architecture is required for a strong B3 analysis

Timing: 3 minutes

This slide highlights the assessment connection. For Quiz 4, expect conceptual questions. For example: why divide dot products by square root of d_k ? The answer is to prevent large values from making softmax too sharp and gradients too small.

Another question: what is the order inside a Transformer encoder block? Self-attention with residual and norm, then feedforward with residual and norm.

Another question: why does a Transformer need positional encoding? Because self-attention alone is permutation-invariant; it does not automatically know the order of tokens.

For the TMA, Q5 should now be completable. For the ECA, fine-tuning DistilBERT will make more sense if you understand today's architecture. You do not need to implement everything from scratch, but you must understand what the model is doing.

Common Student Errors

- Error 1: Conflating "attention" with "the Transformer" — attention predates the Transformer by 3 years; the Transformer is an architecture built on self-attention
- Error 2: Thinking the context vector c_t replaces the decoder hidden state — it augments it, doesn't replace it
- Error 3: Misreading the attention matrix — rows are decoder steps, columns are encoder positions
- Error 4: Thinking multi-head attention splits the sequence across heads — all heads see all positions, just with different learned projections
- Error 5: Confusing encoder self-attention (Q,K,V all from the same sequence) with decoder cross-attention (Q from decoder, K/V from encoder)
- Error 6: Omitting residual connections when drawing the encoder block — they are what makes deep Transformers trainable

Timing: 4 minutes

Let us end with common mistakes. First, do not confuse attention with the Transformer. Attention came before the Transformer. The Transformer is an architecture built mainly on self-attention.

Second, the context vector in attention does not replace the decoder hidden state. It augments the decoder state with source information.

Third, when reading an attention matrix, remember: rows are decoder steps or target words, and columns are encoder positions or source words.

Fourth, multi-head attention does not split the sequence across heads. Every head can see all positions, but each head uses different learned projections.

Fifth, encoder self-attention and decoder cross-attention are different. In encoder self-attention, Q, K, and V come from the same sequence. In cross-attention, Q comes from the decoder, while K and V come from the encoder.

Finally, do not omit residual connections when drawing the encoder block. They are essential for training deep Transformers.